Event Manager App Report

Contents

Three-tier architecture diagram

Figure 1 represents an architecture diagram for this event manager app. The architecture diagram contains three tiers:

1. Data Tier

This tier contains the data tables for events, bookings and site settings. The application tier interacts with the data tier through SQL queries. Although there is a range of SQL queries, this diagram uses a high-level abstraction to categorize SQL queries into two main groups (SELECT and UPDATE). The SELECT group represents queries that access data from the tables. The UPDATE group represents queries that modify data in the tables.

2. Application Tier

This tier receives HTTP requests from the presentation tier and queries the data tables in the data tier through SQL. The "index.js" file contains logic to handle entry point routes. The "routes/organiser.js" file contains logic to handle routes for organiser functionality. The "routes/attendee.js" file contains logic to handle routes for attendee functionality.

3. Presentation Tier

This tier renders HTML pages in the browser from EJS templates. The rendered pages include the main home page, the organiser home page, the organiser edit event page, the organiser site settings page, the attendee home page and the attendee event page. To render dynamic content, it interacts with the application tier through HTTP requests.

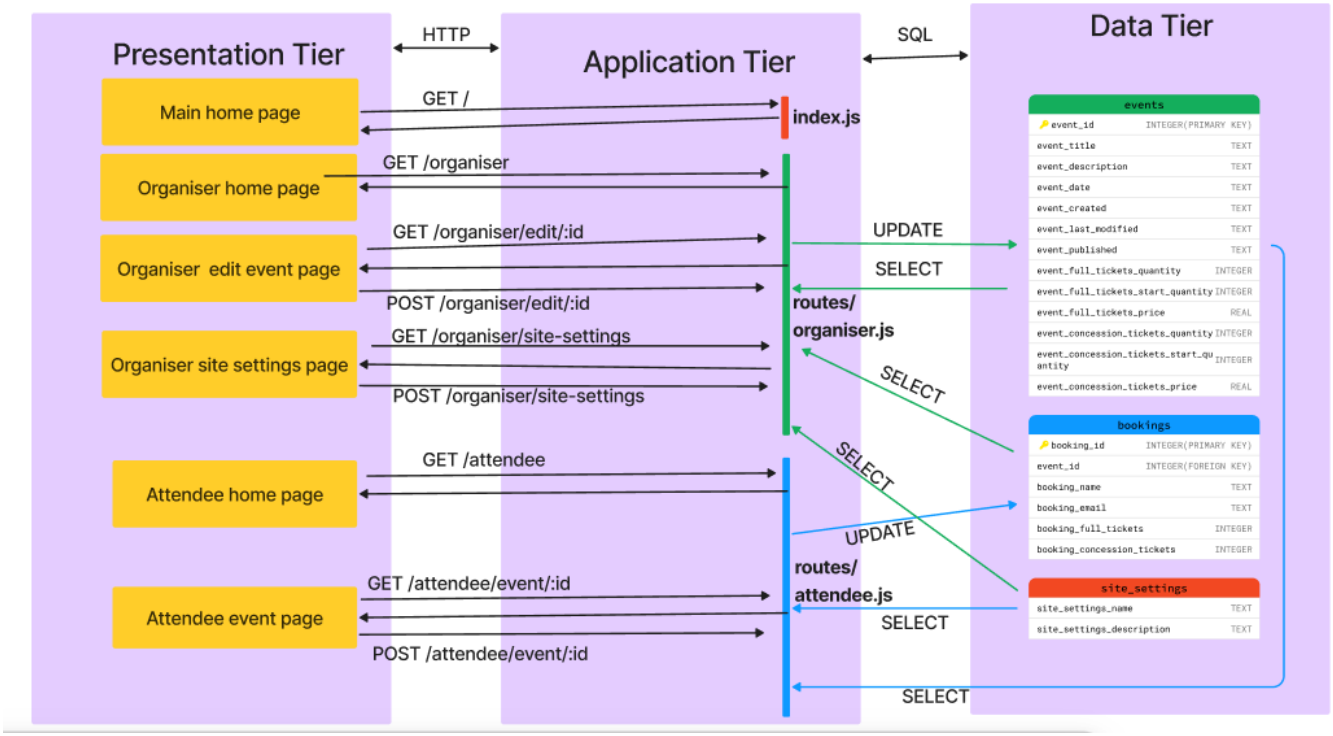

Figure 1: Three-tier architecture diagram

Entity Relationship Diagram

Figure 2 shows the entity relationship diagram for this event manager app. The data model contains the following tables/entities:

1. **Events**

This table stores event details (title, description, date). Additionally, it stores timestamps for when the event was created, last modified and published (if the event is not published this field is set to 0). For each ticket type (full price and concession), it stores the ticket price, the total number of tickets and the remaining number of tickets. A unique integer 'event_id' is set as the primary key for each event.

2. **Bookings**

This table stores information about event bookings: attendee name, attendee email, number of full price tickets booked, and number of concession price tickets booked. The primary key 'event_id' of the events table acts as a foreign key for the bookings table. Therefore, each booking corresponds to one event. One event can have multiple corresponding bookings (or no bookings). If an event from the events table is deleted, all of its corresponding bookings in the bookings table are deleted. The one-to-many optional relationship between events and bookings is depicted through the crow's feet notation on the entity relationship diagram.

3. **Site Settings**

This table stores the site settings (site name and site description). This table does not have any relationship with the other tables/entities because site settings are global and independent of event or booking information. Therefore, the entity relationship diagram shows no relationship of the site settings table with other entities.

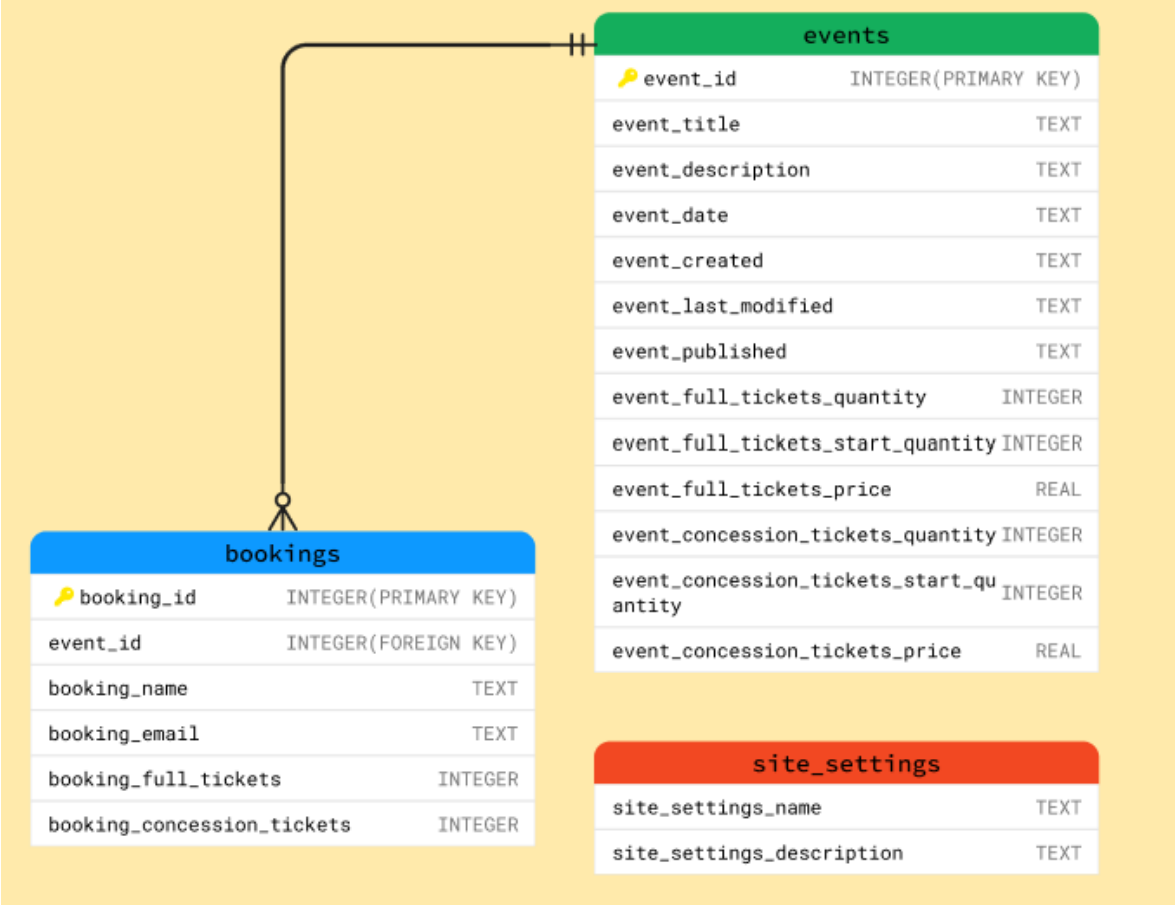

Figure 2: Entity Relationship Diagram

Extension

I chose the first extension option: adding extra organiser and attendee functionality. To implement this, I added the following functionality to the app.

First Feature: Displaying the number of remaining tickets

On the organiser home page, the number of remaining tickets (full price and concession) are displayed for each published event. On the attendee event page, the booking form displays the number of remaining tickets. The following steps were taken to achieve this functionality:

- For both ticket types (full price and concession), a field in the 'events' table was created to store initial ticket quantity, and another field was created to store remaining ticket quantity.
- When an event is created, the total ticket quantities and remaining ticket quantities are the same.
- When an attendee books an event, the remaining ticket quantity for both full price and concession tickets are dynamically updated. The route which handles the POST request for the attendee event booking form runs a query which deducts the booked ticket quantities from the remaining ticket quantities. Figure 3 shows a code snippet of the route and the query for updating remaining tickets.

```
router.post('/event/:id', (req, res, next) => {

    // Query for updating the remaining ticket quantities in the 'events' table
    // for the event which has the same id as the request parameter id.
    // It deducts the number of booked full-price/concession tickets from the number of
    // remaining full-price/concession tickets
    const remainingTicketsQuery = `
        UPDATE events
        SET
        event_full_tickets_quantity = event_full_tickets_quantity - ?,
        event_concession_tickets_quantity = event_concession_tickets_quantity - ?
        WHERE
        event_id = ?
    `;
```

Figure 1: Code snippet from "routes/attendee.js" line 70 onwards

Second Feature: Dropdown menu to display booking details

On the organiser home page, a dropdown menu for each published event displays the event's booking details. For each booking, the attendee's name and email are displayed along with the number of booked full price and concession price tickets. The following steps were taken to achieve this functionality:

- The booking information for each booking corresponding to published events is passed to the EJS template of the organiser home page.
- The organiser home page template uses an HTML select element to insert the booking information in the options of a dropdown menu. Figure 4 shows a code snippet from the EJS template, which inserts booking information into options of a select element.

Line 71 onwards in the file "views/organiser/home.ejs" contains the code for.

```
<!-- Setting the selected index to 0 so that the first option
    "Attendees" is always selected. I learnt how to keep an option selected
    from https://www.w3schools.com/jsref/prop_select_selectedindex.asp -->
<select onchange="this.selectedIndex = 0;">

    <!-- The first option contains the text 'Attendees' -->
    <option>Attendees</option>

    <!-- Iterating over each item in event.bookings -->
    <% event.bookings.forEach(booking => { %>
        <!-- Storing the attendee name in an option -->
        <option><%= booking.booking_name %></option>
        <!-- Storing the attendee email in an option -->
        <option><%= booking.booking_email %></option>
        <!-- Storing the number of booked full price tickets in an option -->
        <option><%= booking.booking_full_tickets %> Full Price Tickets</option>
        <!-- Storing the number of booked concession tickets in an option -->
        <option><%= booking.booking_concession_tickets %> Concession Price Tickets</option>
        <!-- Leaving an empty option to give space between different bookings -->
        <option> </option>
    <% }); %>
</select>
```

Figure 2: Code snippet from the file "views/organiser/home.ejs" line 68 onwards

Third feature: Unpublish button

On the organiser home page, each published event has an unpublish button. When this button is pressed, the published event's state is set to unpublished, and it appears under the draft events. Additionally, all the booking information corresponding to the event is deleted, and the remaining number of tickets is reset to the total number of tickets.

To achieve this functionality, a route was defined to handle POST requests for when the unpublish button is pressed. Figure 5 displays the code snippet of the route definition. This route runs queries to achieve the following:

- Updates the event's published timestamp in the 'events' table to 0, to indicate that the event has not been published
- Deletes all entries in the bookings table corresponding to the event.
- Resets the remaining number of tickets to the original number of tickets (in the 'events' table).

```
// Route for POST requests when an event unpublish button is pressed
// (event corresponding to the request parameter id)
// It updates the event's published timestamp to 0
// indicating that the event is not published.
// It also deletes all bookings for the event.
// Additionally, it resets the remaining number of
// tickets to the original number of tickets.
// Then, it redirects to the organiser home page.
router.post('/unpublish/:id', (req, res, next) => {

    // Query for updating the event published timestamp in the 'events' table
    // for the event which has the same id as the request parameter id.
    // It sets the published timestamp to 0 to indicate that the event is not published.
    const unpublishQuery = 'UPDATE events SET event_published = 0 WHERE event_id = ?';

    // Query for deleting entries in the 'bookings' table,
    // for the event which has the same id as the request parameter id.
    const deleteBookingsQuery = 'DELETE FROM bookings WHERE event_id = ?';

    // Query for updating the remaining ticket quantities in the 'events' table
    // for the event which has the same id as the request parameter id.
    // For full price tickets and concession tickets, it resets the
    // quanitites to the origninal ticket quantities.
    const resetTicketsQuery = `
        UPDATE events SET
        event_full_tickets_quantity = event_full_tickets_start_quantity,
        event_concession_tickets_quantity = event_concession_tickets_start_quantity
        WHERE event_id = ?
    `;

    // executing the query to set the event published timestamp to 0
    global.db.get(unpublishQuery, req.params.id, (err, event) => {
        if (err) return next(err);

        // executing the query to delete bookings corresponding to the event
        global.db.run(deleteBookingsQuery, req.params.id, err => {
            if (err) return next(err);

            // executing the query to reset the remaining ticket quantities to
            // original ticket quanitities
            global.db.run(resetTicketsQuery, req.params.id, err => {
                if (err) return next(err);

                // redirecting to the organiser home page
                res.redirect('/organiser');
            });
        });
    });
});
```

Figure 3: Code snippet from the file "routes/organiser.js" line 237 onwards.